



HoàngYell
I'm crazy, lazy and busy!







GitNexus: Knowledge Graph Giúp AI Agents Thực Sự Hiểu Codebase Của Bạn

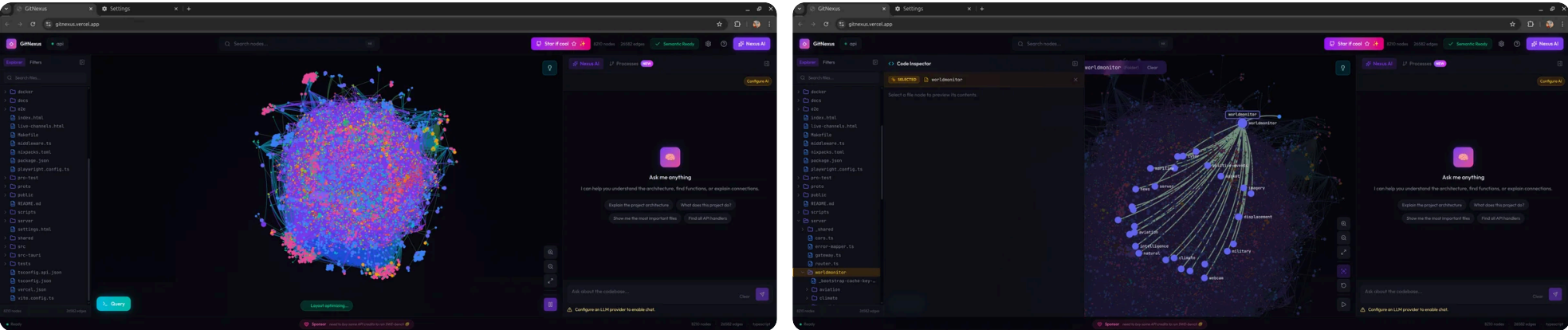
GitNexus index toàn bộ codebase thành knowledge graph — mọi dependency, call chain, cluster, và execution flow — rồi cung cấp qua MCP tools để AI agents không bỏ sót code nào.

📅 Feb 28, 2026 ⌚ 9 phút 🌐 English

⚡ TL;DR

Gitnexus là một công cụ MCP giúp AI agent hiểu rõ codebase của bạn hơn, từ đó viết code tốt hơn. Ngoài ra, nó còn cung cấp giao diện dạng đồ thị tại [đây](#), giúp bạn trực quan hóa và hiểu rõ bản chất hoạt động của hệ thống.

- **Nó giải quyết gì:** Xây dựng knowledge graph của codebase và expose qua MCP tools để AI agents hiểu impact của mỗi thay đổi
- **Tại sao quan trọng:** Nếu không có nó, AI refactor code bề ngoài tốt nhưng vô tình break 47 functions phụ thuộc mà nó không thấy
- **Dùng cho ai:** Developers dùng Cursor, Claude Code, Windsurf hay bất kỳ AI coding assistant nào
- **Khác biệt chính:** Tính toán trước graph intelligence (clustering, execution flow, blast radius) thay vì hy vọng LLM dò đủ



Phần 1: Nền Tảng – Mô Hình Tư Duy

Tưởng tượng bạn là bác sĩ phẫu thuật chuẩn bị mổ. Bạn có X-quang thấy xương, nhưng không thấy dây thần kinh, mạch máu, hay cách chúng kết nối. Bạn cắt một nhát – và trúng động mạch mà không ai đề cập.

Đó chính xác là điều xảy ra khi AI agents chỉnh sửa code ngày nay.

Các tool như Cursor, Claude Code, Windsurf, và Cline là những trình soạn thảo code cực kỳ mạnh. Nhưng chúng có chung một điểm mù cốt lõi: chúng không thực sự hiểu *cấu trúc* codebase của bạn. Chúng thấy files, thấy functions, nhưng không thấy mạng lưới dependencies vô hình kết nối mọi thứ.

Đây là pattern thất bại điển hình:

1. Bạn yêu cầu AI refactor `UserService.validate()`
2. AI chỉnh sửa hoàn hảo – trong phạm vi riêng lẻ
3. AI không biết 47 functions phụ thuộc vào return type của nó
4. **Breaking changes** được push lên production

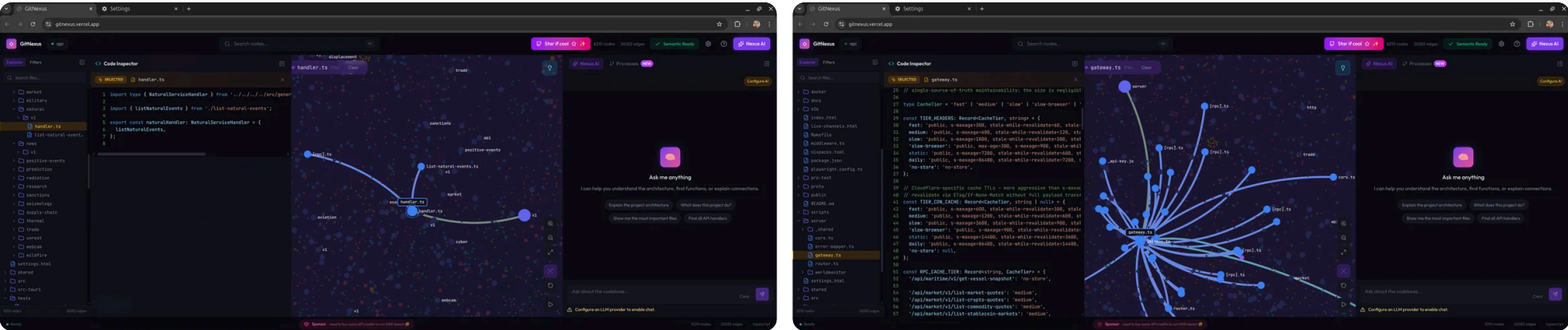
GitNexus giải quyết vấn đề này bằng cách xây dựng một **knowledge graph** hoàn chỉnh của codebase – mọi function call, import, class inheritance, và execution flow – rồi expose qua smart tools thông qua **Model Context Protocol (MCP)**.

Hình dung thế này:

“

Không có GitNexus: AI agent của bạn di chuyển trong codebase như khách du lịch với bản đồ tên đường.

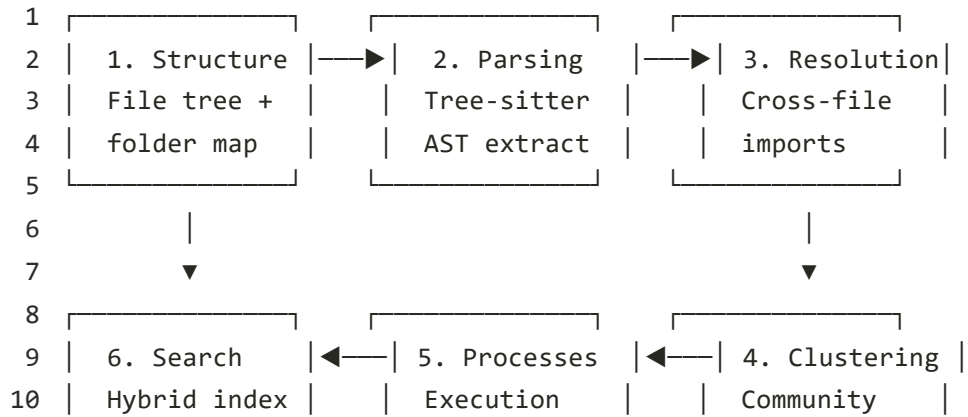
Có GitNexus: AI agent di chuyển như dân địa phương biết mọi đường tắt, ngõ cụt, và đường một chiều.



Phần 2: Khám Phá – GitNexus Xây Dựng “Bộ Não” Như Thế Nào

Pipeline Index Đa Giai Đoạn

Khi bạn chạy `npx gitnexus analyze`, một quá trình đáng chú ý xảy ra bên trong. GitNexus xử lý codebase qua pipeline 6 giai đoạn:



11	BM25+Vector		flow tracing		detection	
12						

Giai đoạn 1 — Structure: Map file tree và quan hệ thư mục. Đây là bộ khung xương.

Giai đoạn 2 — Parsing: Sử dụng [Tree-sitter](#) để trích xuất mọi function, class, method, và interface từ **11 ngôn ngữ**: TypeScript, JavaScript, Python, Java, C, C++, C#, Go, Rust, PHP, và Swift.

Giai đoạn 3 — Resolution: “Phép thuật” xảy ra ở đây. GitNexus resolve imports và function calls *xuyên files* với logic nhận diện ngôn ngữ. Nó không chỉ biết `auth.ts` tồn tại — nó biết `handleLogin()` trong `auth.ts` gọi `validate()` trong `user.ts` với độ tin cậy 90%.

Giai đoạn 4 — Clustering: Nhóm các symbols liên quan thành communities chức năng sử dụng graph algorithms qua [Graphology](#). Các auth functions, database layer, và API routes tự nhiên cluster với nhau.

Giai đoạn 5 — Processes: Trace execution flows từ entry points xuyên suốt call chains. Nó map ra “LoginFlow” là process 7 bước từ route handler → validation → database → response.

Giai đoạn 6 — Search: Xây dựng hybrid search index kết hợp BM25 (keyword), semantic embeddings (qua HuggingFace transformers.js), và Reciprocal Rank Fusion để truy xuất nhanh.

Đổi Mới Cốt Lõi: Precomputed Intelligence

Các cách tiếp cận Graph RAG truyền thống đổ raw graph edges cho LLM rồi hy vọng nó dò đủ. GitNexus **tính toán trước** ngay khi index — clustering, tracing, confidence scoring — để mỗi tool call trả về context *hoàn chỉnh* trong một query duy nhất.

Điều này có nghĩa:

- **LLM không thể bỏ sót context** — nó đã sẵn trong tool response
- **Tiết kiệm tokens** — không cần chuỗi 10 queries để hiểu một function
- **Dân chủ hóa models** — LLM nhỏ hơn cũng hoạt động tốt vì tools lo phần nặng

Tech Stack

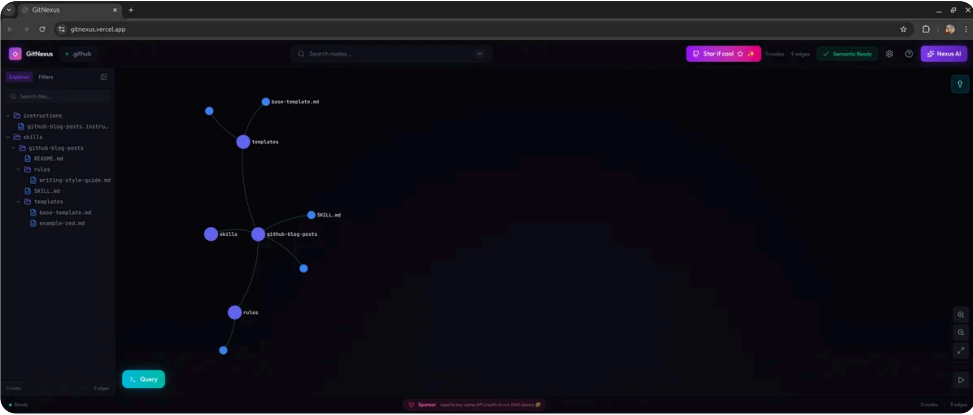
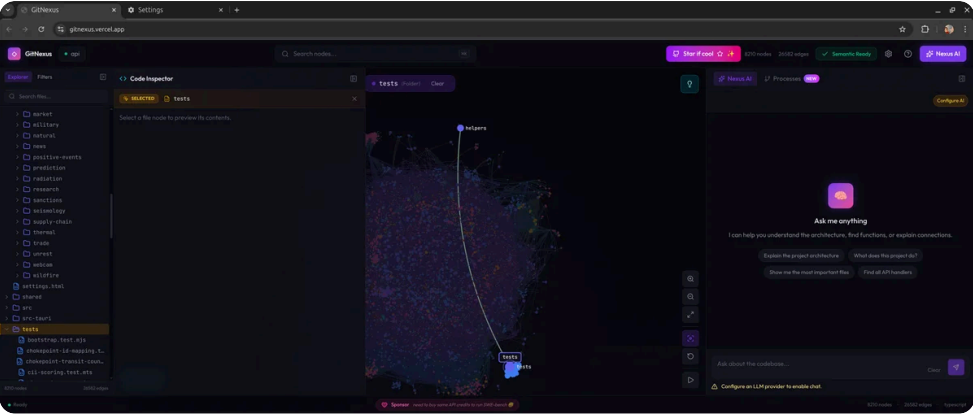
GitNexus chạy ở hai chế độ, mỗi cái với tech phù hợp:

Layer	CLI (Local)	Web (Browser)
Parsing	Tree-sitter native	Tree-sitter WASM
Database	KuzuDB native	KuzuDB WASM
Embeddings	transformers.js (GPU/CPU)	transformers.js (WebGPU/WASM)
Agent Interface	MCP (stdio)	LangChain ReAct agent
Visualization	—	Sigma.js + Graphology (WebGL)

Mọi thứ lưu trong [KuzuDB](#), embedded graph database có hỗ trợ vector — không cần database server bên ngoài.

Phần 3: Chẩn Đoán — GitNexus Thực Sự Làm Gì Cho Developers

7 Tools Cho AI Agents “Tia X” Xuyên Code



Khi bạn kết nối GitNexus qua MCP với editor, AI agent của bạn được truy cập 7 tools mạnh mẽ:

1. impact — Phân Tích Blast Radius

Trước khi chạm vào code, hỏi: “Cái gì sẽ vỡ?”

```
1 impact({target: "UserService", direction: "upstream", minConfidence: 0.8})
2
3 TARGET: Class UserService (src/services/user.ts)
4
5 UPSTREAM (phụ thuộc vào nó):
6   Depth 1 (SẼ VỠ):
7     handleLogin [CALLS 90%] -> src/api/auth.ts:45
8     handleRegister [CALLS 90%] -> src/api/auth.ts:78
9     UserController [CALLS 85%] -> src/controllers/user.ts:12
10  Depth 2 (CÓ THỂ ẢNH HƯỞNG):
11    authRouter [IMPORTS] -> src/routes/auth.ts
```

Giống như có một senior engineer đã thuộc lòng toàn bộ codebase nói: “Nếu bạn thay đổi UserService, 4 thứ này SẼ vỡ, và 2 thứ này CÓ THỂ vỡ.”

2. query — Tìm Kiếm Theo Process

Không chỉ “tìm files chứa X”, mà “tìm các processes và execution flows liên quan đến X”:

```
1 query({query: "authentication middleware"})
2
3 processes:
4   - summary: "LoginFlow"
5     priority: 0.042
6     symbol_count: 4
7     process_type: cross_community
8     step_count: 7
9
10 process_symbols:
11   - name: validateUser
12     type: Function
13     filePath: src/auth/validate.ts
14     process_id: proc_login
15     step_index: 2
```

3. context — Góc Nhìn 360° Về Symbol

Lấy bức tranh toàn diện của bất kỳ symbol nào — ai gọi nó, nó gọi gì, và tham gia processes nào:

```
1 context({name: "validateUser"})
2
3 incoming:
4   calls: [handleLogin, handleRegister, UserController]
5   imports: [authRouter]
6
7 outgoing:
8   calls: [checkPassword, createSession]
```



```
9
10 processes:
11   - name: LoginFlow (step 2/7)
12   - name: RegistrationFlow (step 3/5)
```

4. detect_changes — Lưới An Toàn Trước Commit

Trước khi commit, hiểu tác động thực sự của thay đổi:

```
1 detect_changes({scope: "all"})
2
3 summary:
4   changed_count: 12
5   affected_count: 3
6   risk_level: medium
7
8 affected_processes: [LoginFlow, RegistrationFlow]
```

5. rename — Rename Phối Hợp Multi-File

Không phải find-and-replace đơn giản, mà rename nhận biết graph — phân biệt giữa function tên `validate` và comment chứa từ “validate”:

```
1 rename({symbol_name: "validateUser", new_name: "verifyUser", dry_run: true})
2
3 files_affected: 5
4 total_edits: 8
5 graph_edits: 6      (độ tin cậy cao)
6 text_search_edits: 2 (cần review cẩn thận)
```

6 & 7. cypher và list_repos

Raw Cypher graph queries cho power users, và khám phá repository cho multi-repo setups.

Use Case Thực Tế: Python Developers

Tưởng tượng bạn đang làm việc trên dự án Django với 200+ models. Bạn cần rename một model field. Không có GitNexus, bạn sẽ:

1. `grep` tên field (bắt luôn comments, strings, matches không liên quan)
2. Trace thủ công serializers, views, và templates
3. Hy vọng không bỏ sót queryset filter nào đó

Với GitNexus: `impact({target: "User.email", direction: "upstream"})` → bản đồ dependency hoàn chỉnh tức thì.

Phần 4: Giải Pháp — Bắt Đầu Như Thế Nào

CLI Quick Start (Khuyến Nghị)

```
1 # Index repository (chạy từ root repo)
2 npx gitnexus analyze
3
4 # Vậy thôi! Lệnh này làm tất cả:
5 # - Index codebase
6 # - Cài đặt agent skills
7 # - Đăng ký Claude Code hooks
8 # - Tạo files AGENTS.md / CLAUDE.md
```

Kết Nối Với Editor

```
1 # Tự động cấu hình MCP cho tất cả editors phát hiện được
2 npx gitnexus setup
3
4 # Hoặc thủ công cho Cursor (~/.cursor/mcp.json):
5 {
6   "mcpServers": {
7     "gitnexus": {
8       "command": "npx",
9       "args": ["-y", "gitnexus@latest", "mcp"]
10    }
11  }
12 }
```

Ma Trận Hỗ Trợ Editor

Editor	MCP	Skills	Hooks	Mức Hỗ Trợ
Claude Code	✓	✓	✓ PreToolUse	Đầy đủ
Cursor	✓	✓	—	MCP + Skills
Windsurf	✓	—	—	MCP
OpenCode	✓	✓	—	MCP + Skills

Web UI (Khám Phá Nhanh)

Không cần cài đặt — chỉ cần truy cập [gitnexus.vercel.app](#). Upload repo hoặc paste GitHub URL. Mọi thứ chạy trong browser — không có code nào gửi tới server.

Bridge Mode

Chạy `gitnexus serve` để kết nối CLI và Web:

```
1 # Start local server
2 gitnexus serve
3
4 # Web UI tự phát hiện — duyệt tất cả repos đã index bằng CLI
5 # không cần upload hay index lại
```

Wiki Generation

Tạo documentation bằng LLM từ knowledge graph:

```
1 gitnexus wiki
2 gitnexus wiki --model gpt-4o
3 gitnexus wiki --force # Tạo lại hoàn toàn
```

Mô Hình Tư Duy Cuối Cùng

Khía Cạnh	Mô Tả
Là gì	Engine knowledge graph index codebases thành graph database có thể query
Tech cốt lõi	Tree-sitter (AST) + KuzuDB (graph DB) + HuggingFace (embeddings)

Khía Cạnh	Mô Tả
Giao diện	7 MCP tools cho AI agents, CLI cho developers, Web UI để khám phá
Insight chính	Precomputed relational intelligence > raw graph traversal
Ngôn ngữ	TypeScript, JavaScript, Python, Java, C, C++, C#, Go, Rust, PHP, Swift
Privacy	Mọi thứ chạy local (CLI) hoặc trong browser (Web). Zero data rời máy bạn
So với DeepWiki	DeepWiki giúp bạn <i>hiểu</i> code. GitNexus giúp bạn <i>phân tích</i> code

GitNexus không thay thế AI coding assistant — nó cho assistant của bạn **trí nhớ chụp ảnh** toàn bộ kiến trúc codebase. Kết quả? Ít breaking changes hơn, refactor thông minh hơn, và AI agents cuối cùng hiểu code mà chúng đang chỉnh sửa.

GitHub: github.com/abhigyanpatwari/GitNexus

Tech GitHub Tech GitHub AI MCP Knowledge Graph Developer Tools

CHIA SẺ BÀI VIẾT



NỘI DUNG LIÊN QUAN

Superpowers: Quy Trình Dạy Agents AI Thành Người Kỉ Luật

BitNet: Kỷ nguyên LLM 1-bit Cuối cùng đã Bắt đầu

Lightpanda: Trình duyệt Headless 'Không tì vết' viết bằng Zig

MiroFish: Diễn tập tương lai trong 'Hộp cát' xã hội AI

0 Comments - powered by utteranc.es

WritePreview

Sign in to comment

Styling with Markdown is supported

Sign in with GitHub